Estruturas de Dados

Skew Heap (Heap Desequilibrada)
Paired Heap (Heap Emparelhada)



2022/2023

Skew Heap

Skew Heap ou Heap Desequilibrada

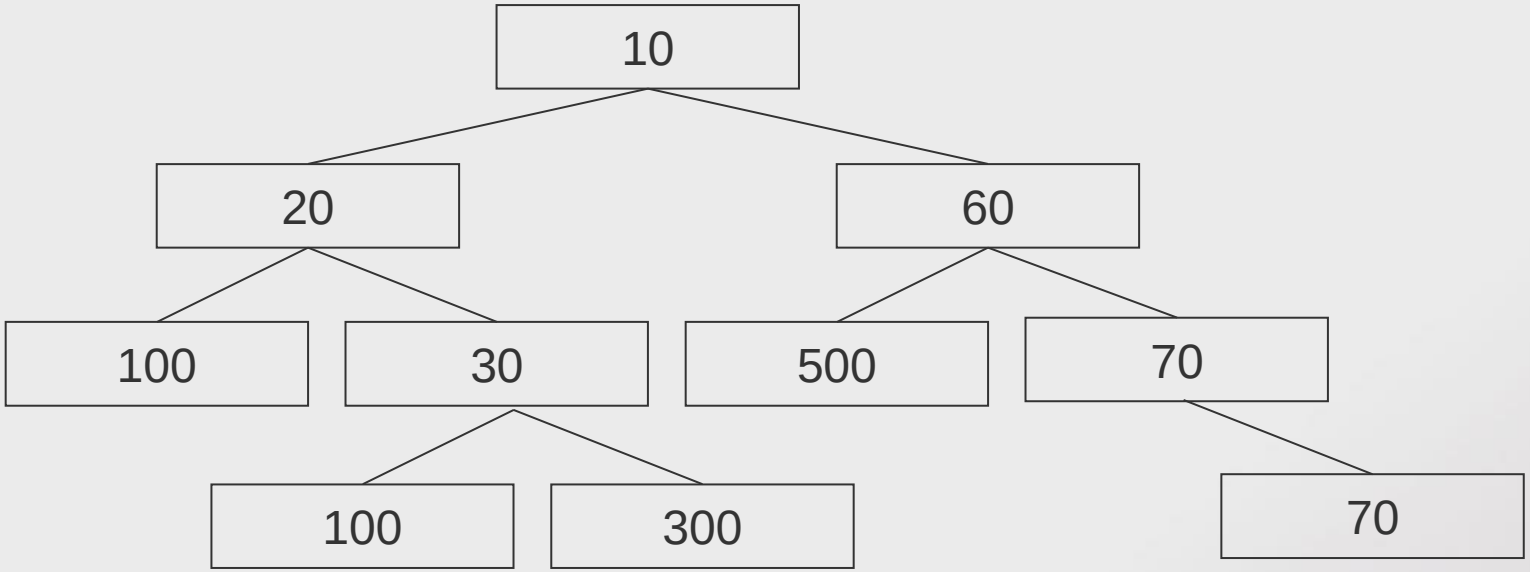
Similar a *Heap* Binária, mas sem condição de equilibrio:

- Verifica propriedade de ordem de *heap*.
- Não é uma ABC.



Skew Heap

- Exemplo





Skew Heap

- As Skew Heaps suportam a operação de junção de duas heaps.
- A junção de *heaps* é uma operação fundamental, dado que todas as outras podem ser feitas recorrendo a ela:
  - **Inserção (X)** – Junção da árvore original com uma árvore contendo apenas o nodo X.
  - **removeMinimo()** – Remove raiz e faz junção das sub-árvores esquerda e direita.
  - **diminuiValor(p, novoValor)** – Separa a subárvore com raiz no nodo *p*, diminui o valor do nodo *p* para *novoValor* e faz a junção das duas árvores resultantes.



Junção de *Heaps*

- Juntar duas *heaps* é fácil…se não nos preocuparmos com a eficiência.

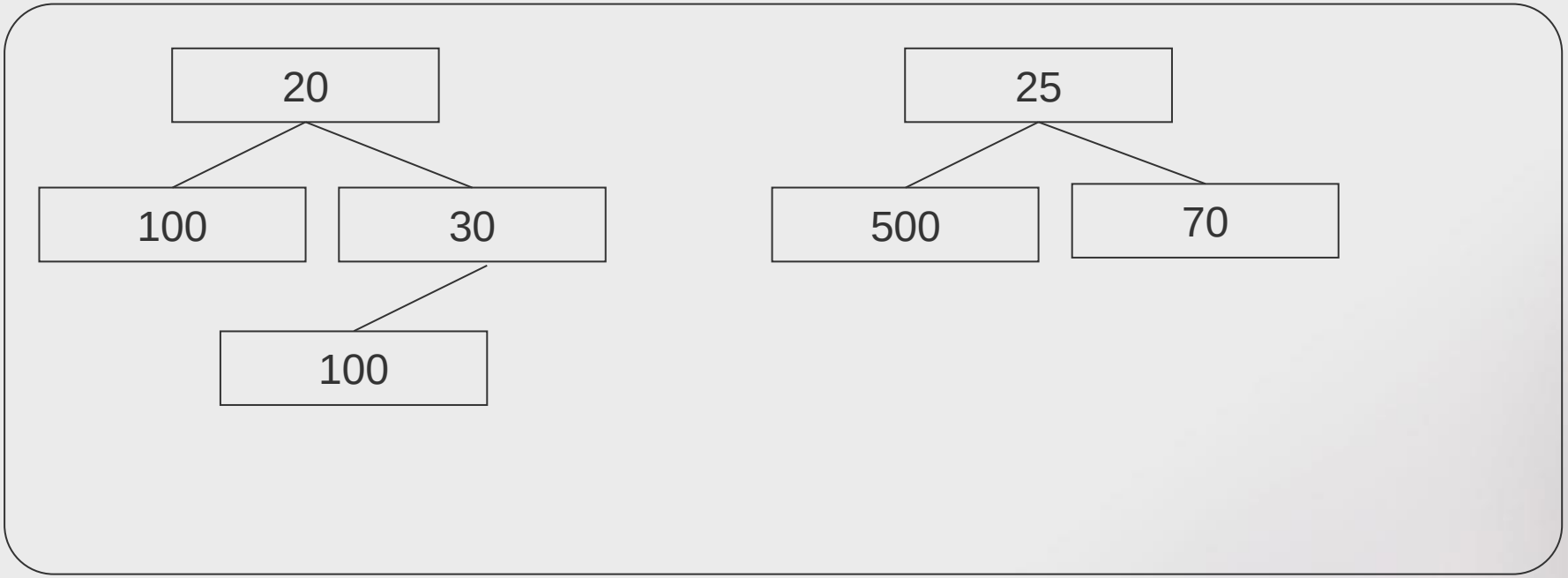
Junção simplista: Recursivamente, o resultado da junção de duas árvores com raizes r1 e r2, com r1≤r2, é dado por:

- A outra sub-árvore, se *r1* ou *r2* estiverem vazias.
- Se *r1≤r2*, a árvore com raiz em *r1* após junção da árvore *r2* com o descendente direito de *r1*.
- Se *r2<r1*, a árvore com raiz em *r2* após junção da árvore *r1* com o descendente direito de *r2.*



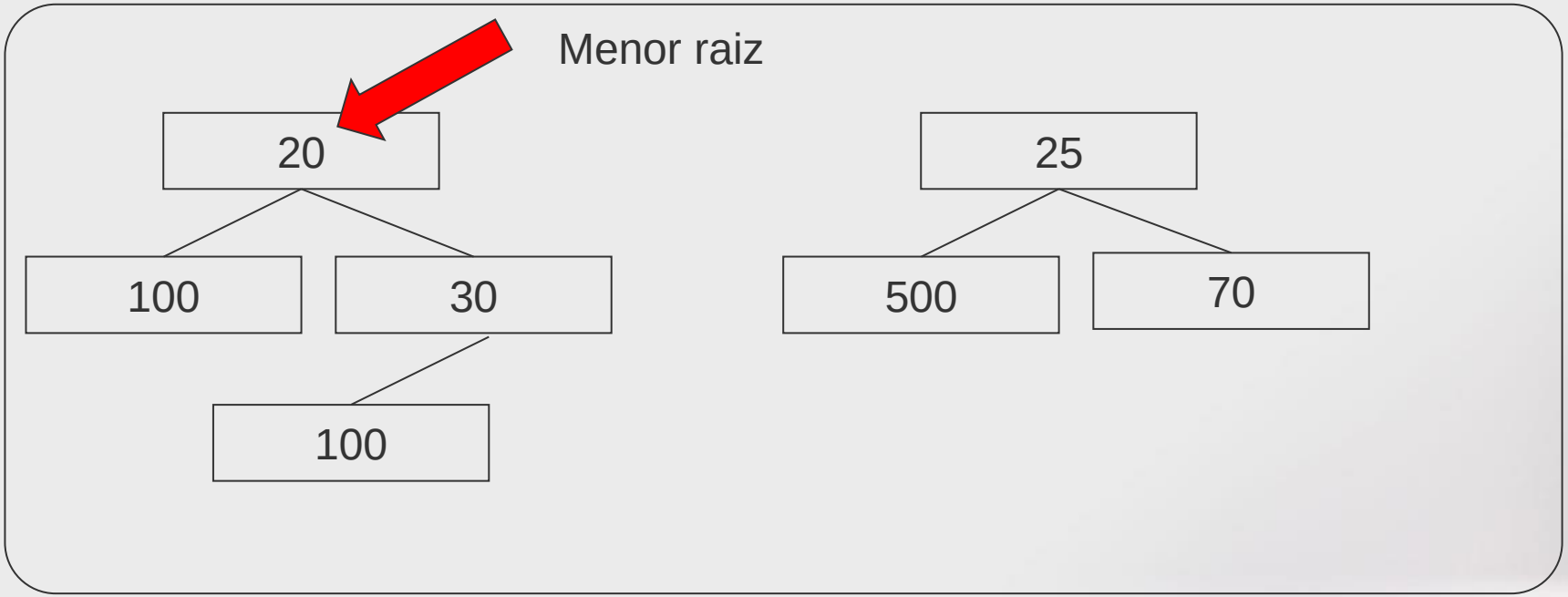
Skew Heap

- Exemplo: Junção simplista de duas heaps



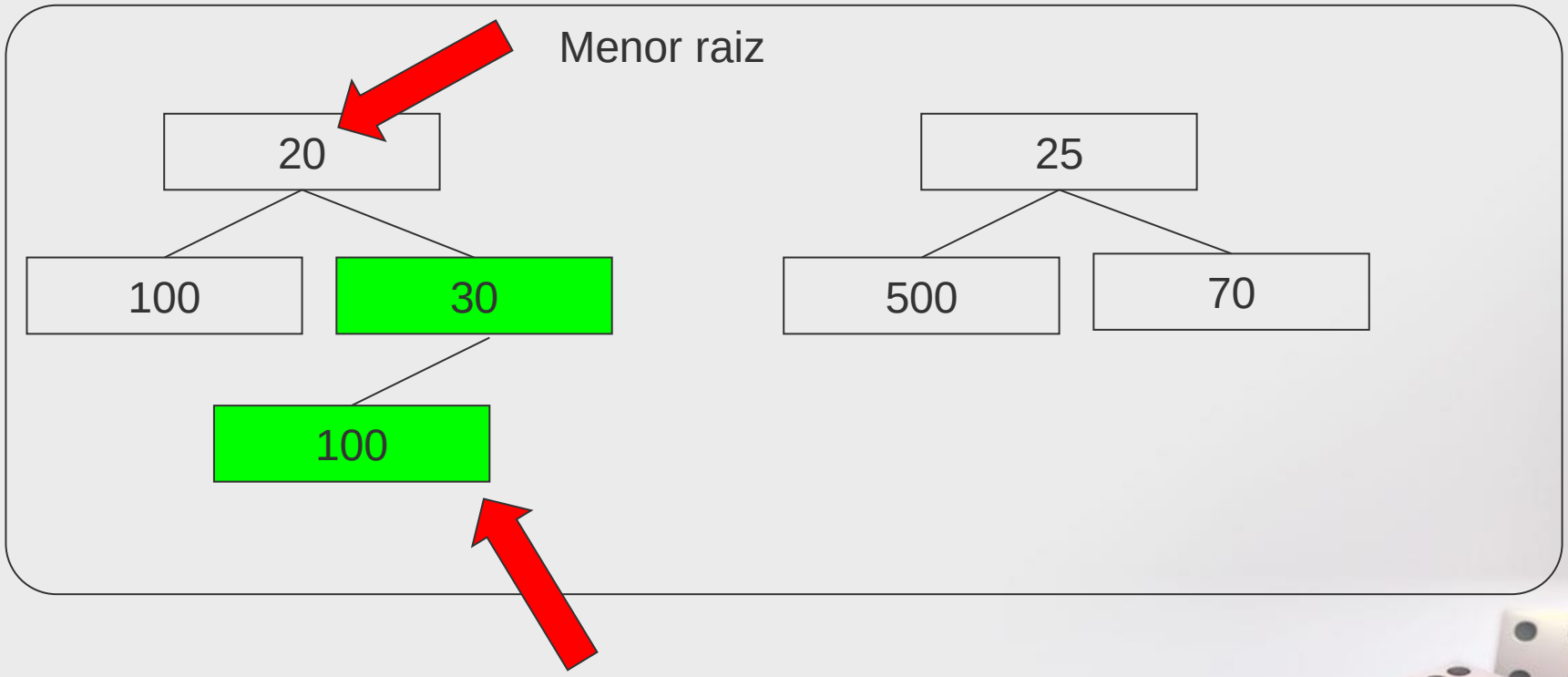
Skew Heap

- Exemplo: Junção simplista de duas heaps



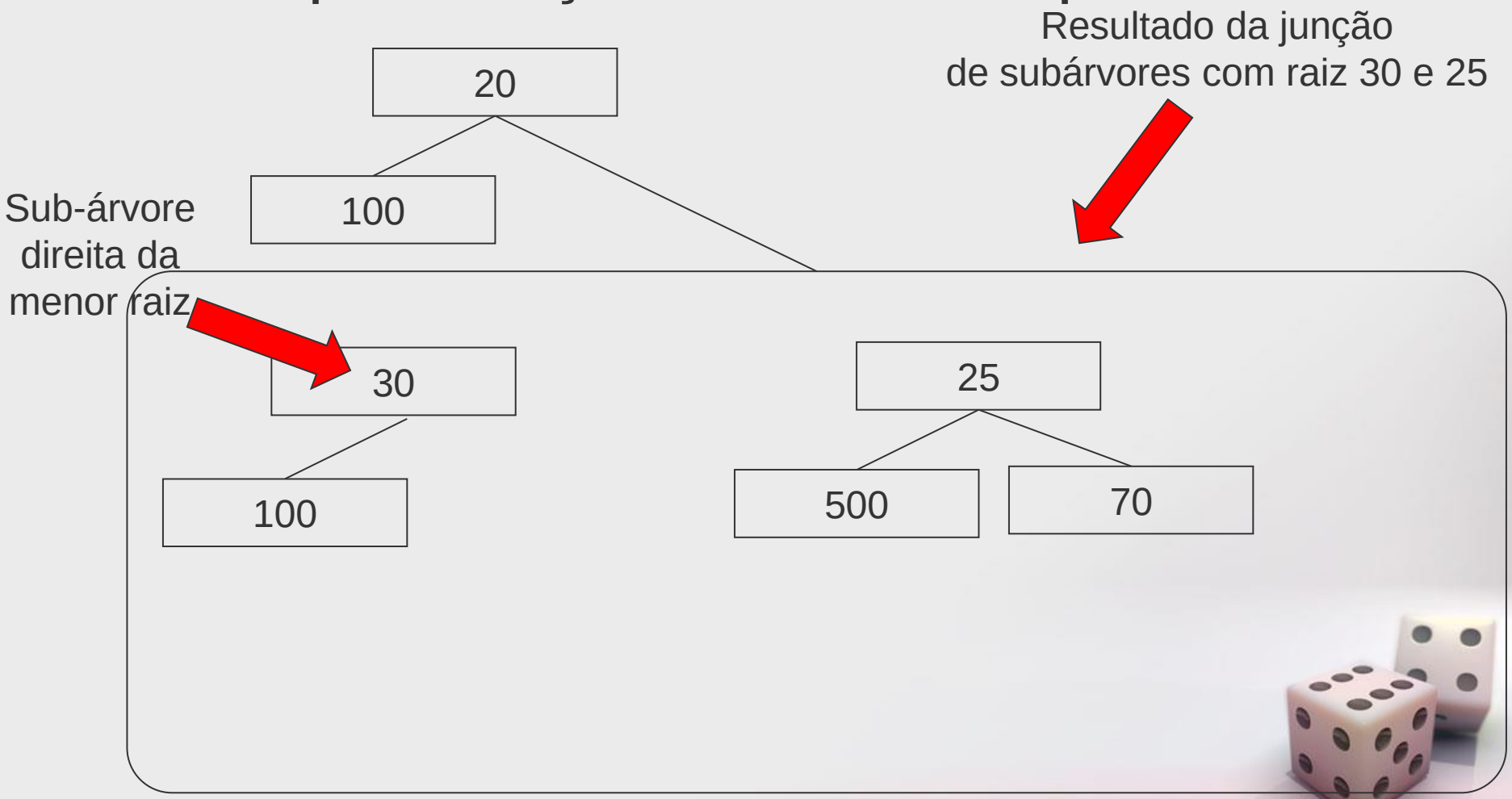
Skew Heap

- Exemplo: Junção de duas heaps



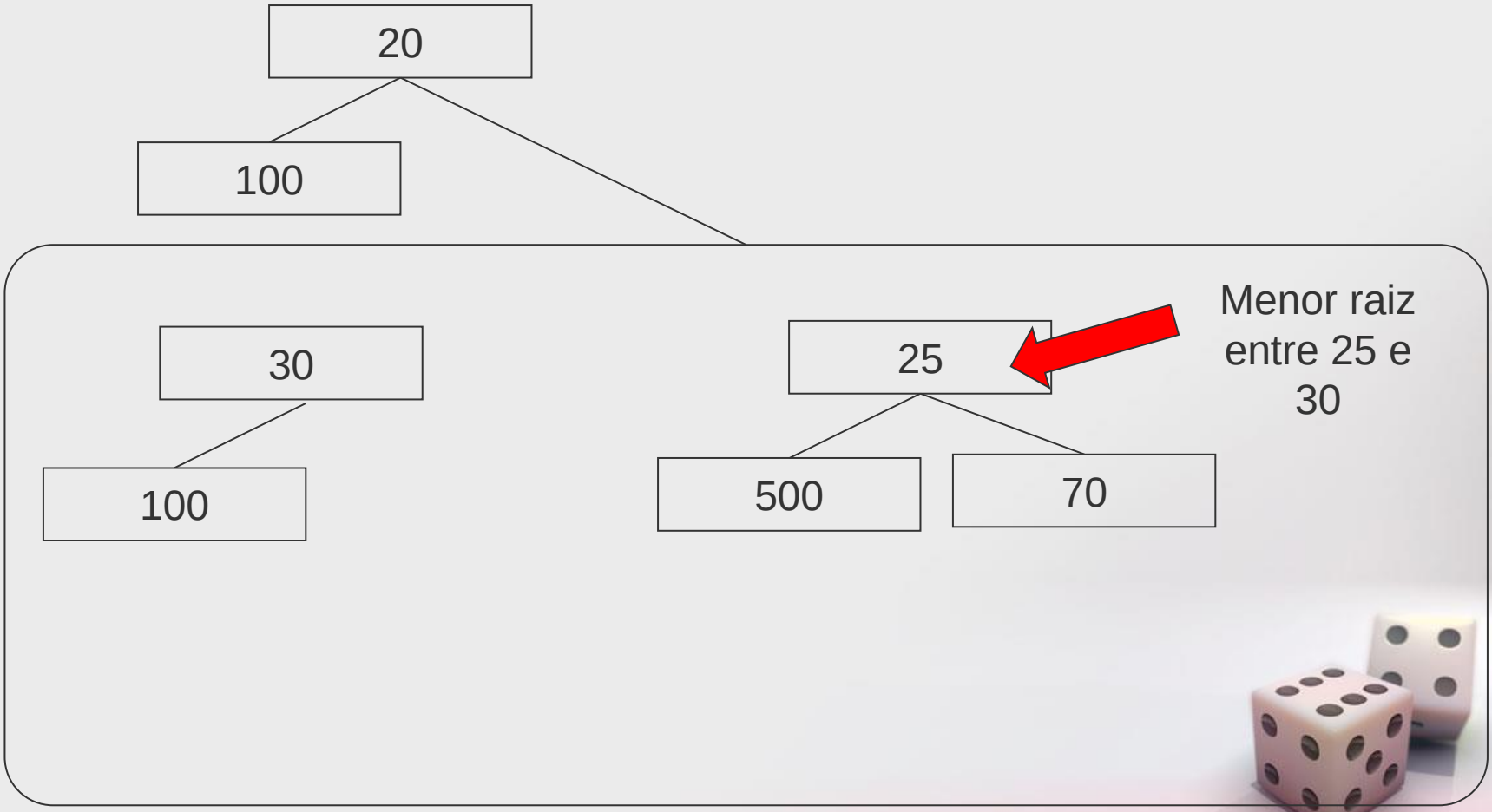
Skew Heap

- Exemplo: Junção de duas heaps



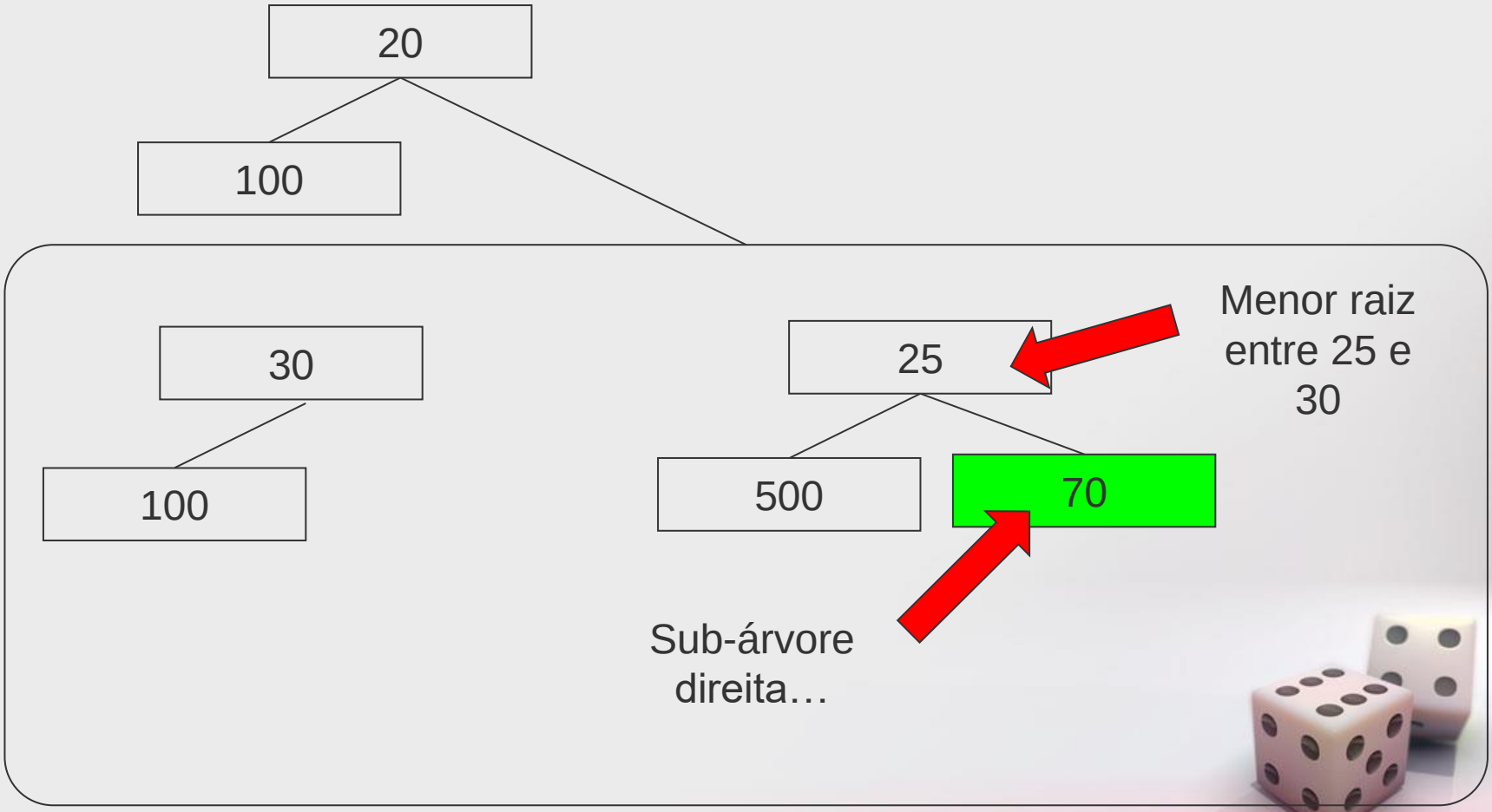
Skew Heap

- Exemplo: Junção simplista de duas heaps



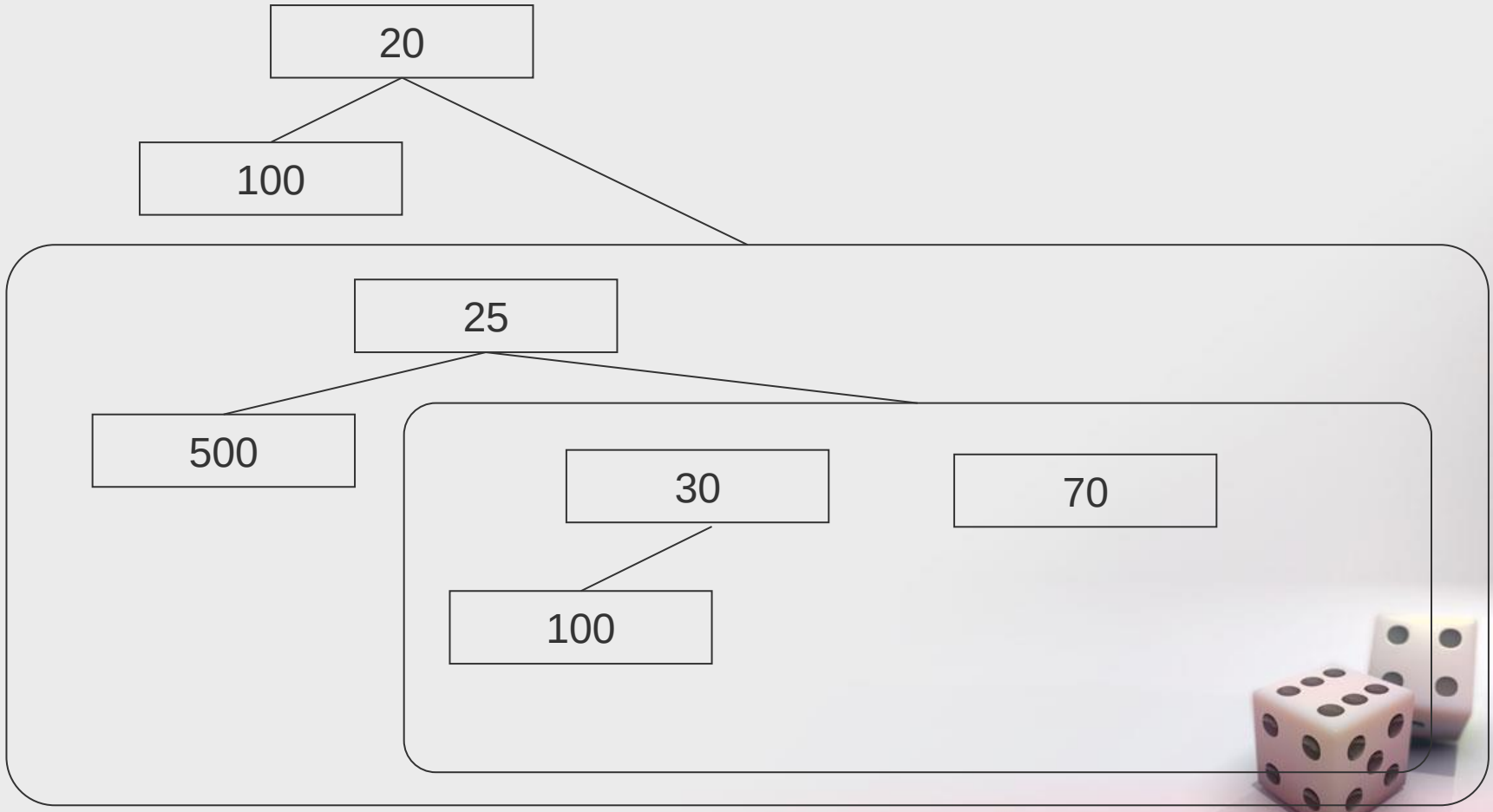
Skew Heap

- Exemplo: Junção simplista de duas heaps



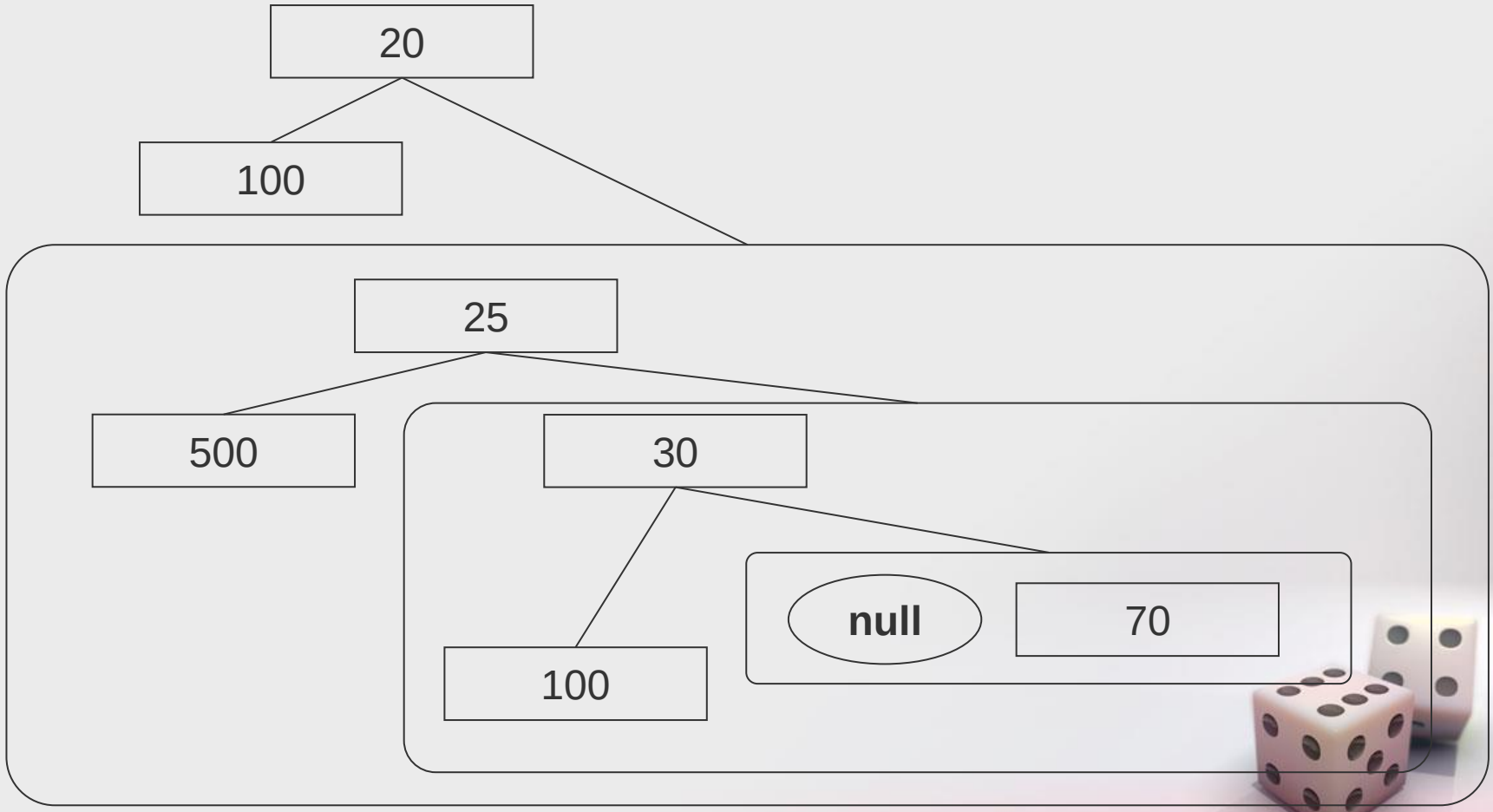
Skew Heap

- Exemplo: Junção simplista de duas heaps



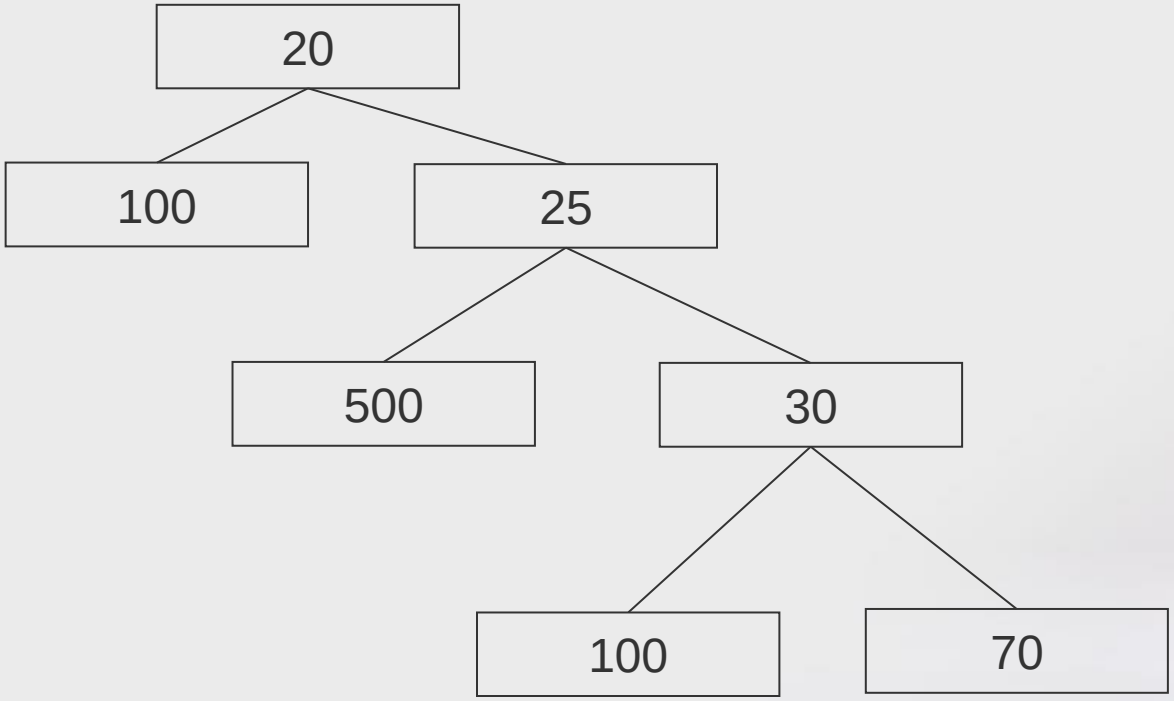
Skew Heap

- Exemplo: Junção simplista de duas heaps



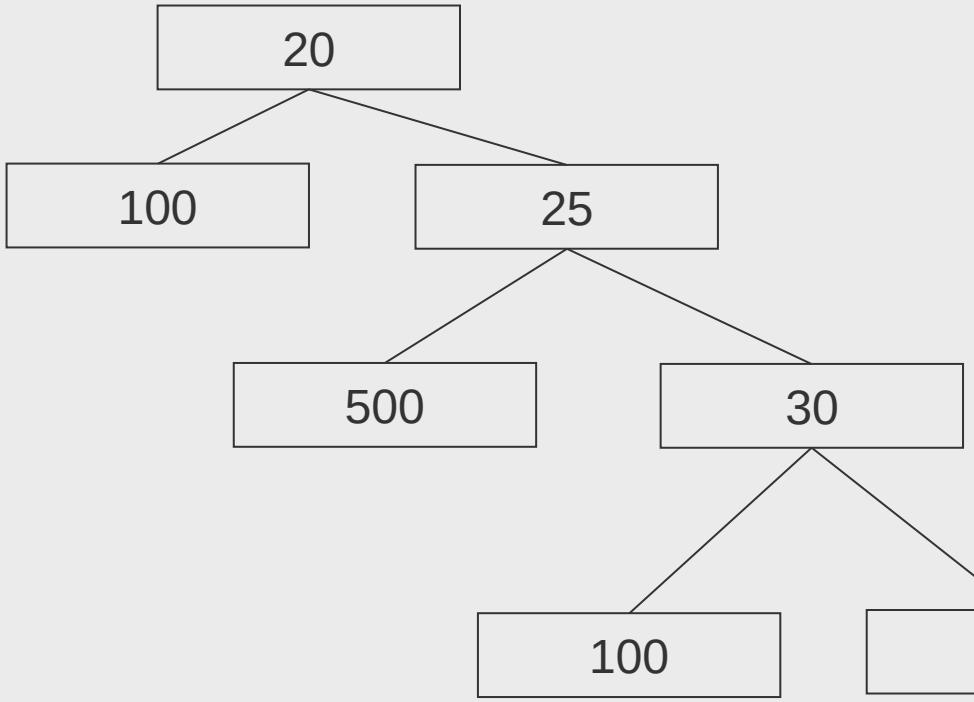
Skew Heap

- Exemplo: Junção simplista de duas heaps:
  - Resultado final



Skew Heap

- Exemplo: Junção simplista de duas heaps:
  - Resultado final



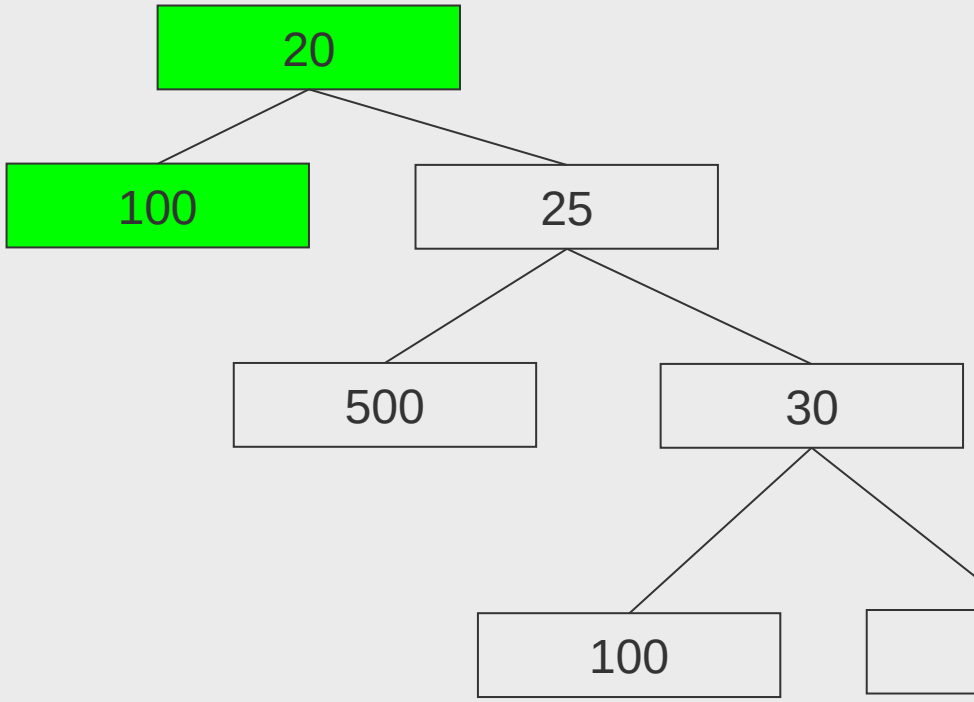
Todos os nodos preservam a sua sub-árvore esquerda original





Skew Heap

- Exemplo: Junção simplista de duas heaps:
  - Resultado final



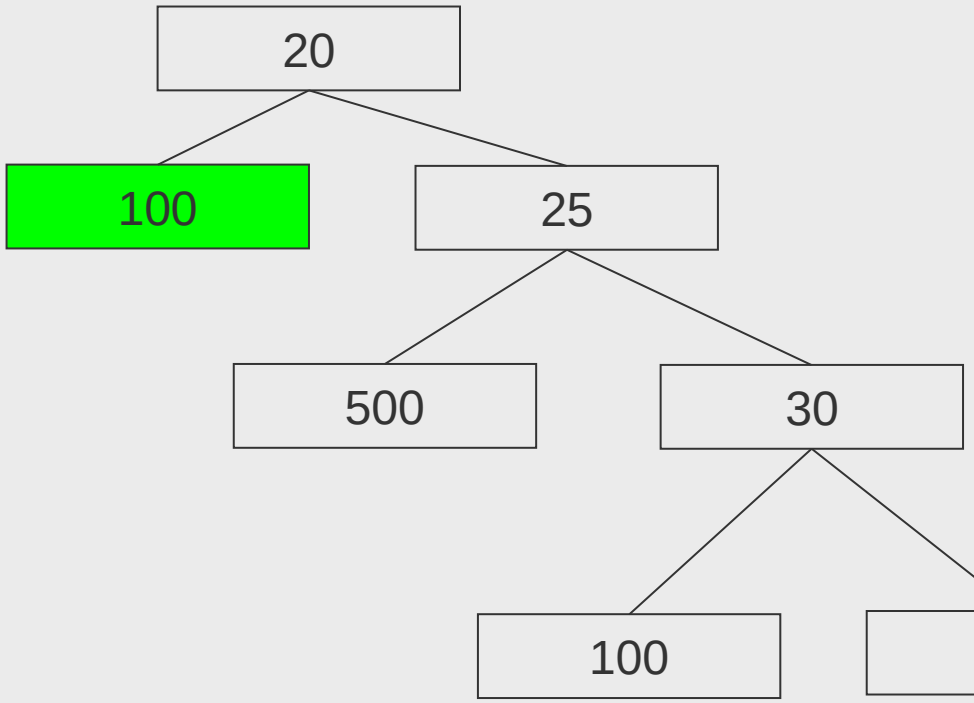
Todos os nodos preservam a
sua sub-árvore esquerda original





Skew Heap

- Exemplo: Junção simplista de duas heaps:
  - Resultado final



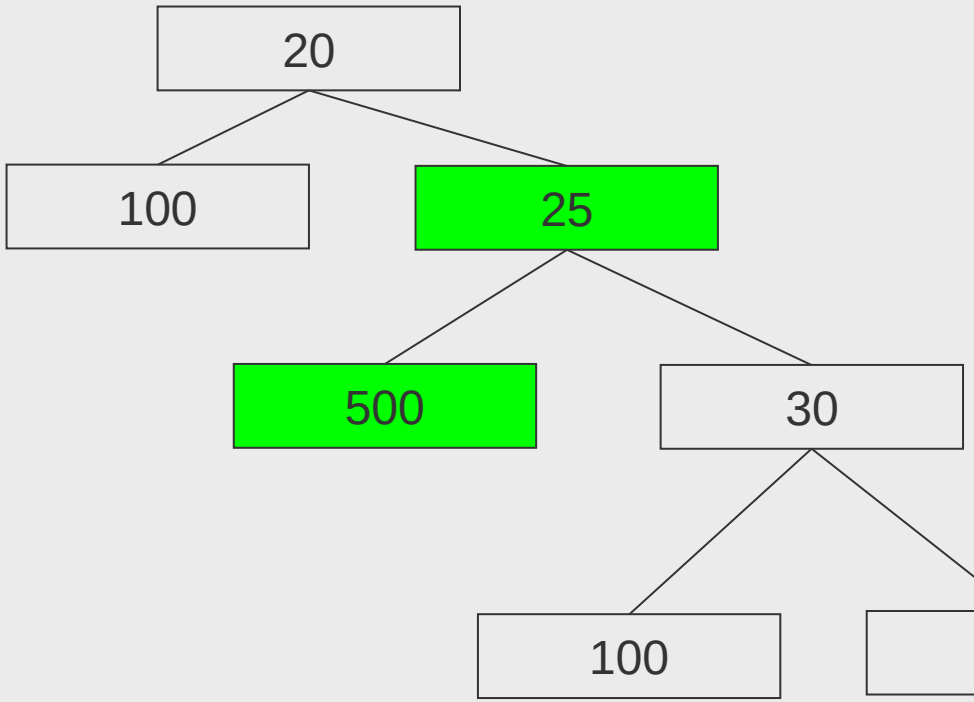
Todos os nodos preservam a
sua sub-árvore esquerda original





Skew Heap

- Exemplo: Junção simplista de duas heaps:
  - Resultado final



Todos os nodos preservam a sua sub-árvore esquerda original



Etc…



Skew Heap



Todos os nodos preservam a sua sub-árvore esquerda original ?!!?!?!?!?!!

-Há algo de muito errado com isso…



Skew Heap

- Vamos ver o que acontece quando inserimos os valores (70, 80, 60) numa àrvore vazia, utilizando a operação de junção “simplista”:
- Inserção de 70

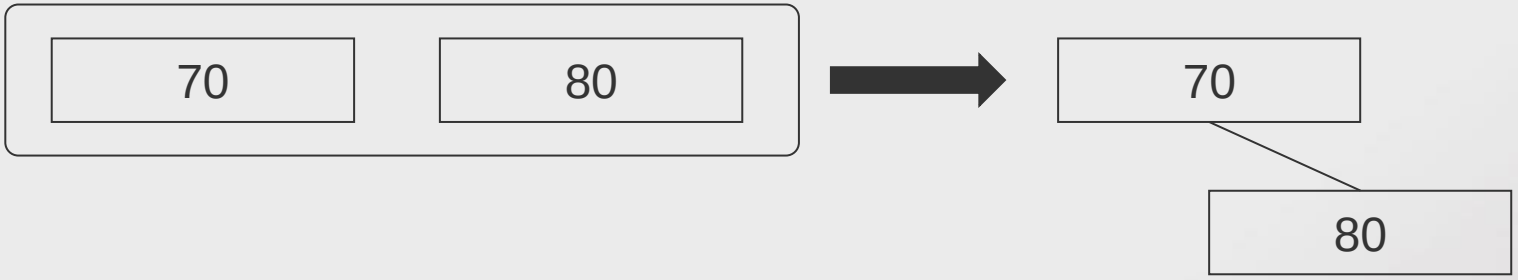




Skew Heap

- Vamos ver o que acontece quando inserimos os valores (70, 80, 60) numa àrvore vazia, utilizando a operação de junção “simplista”:

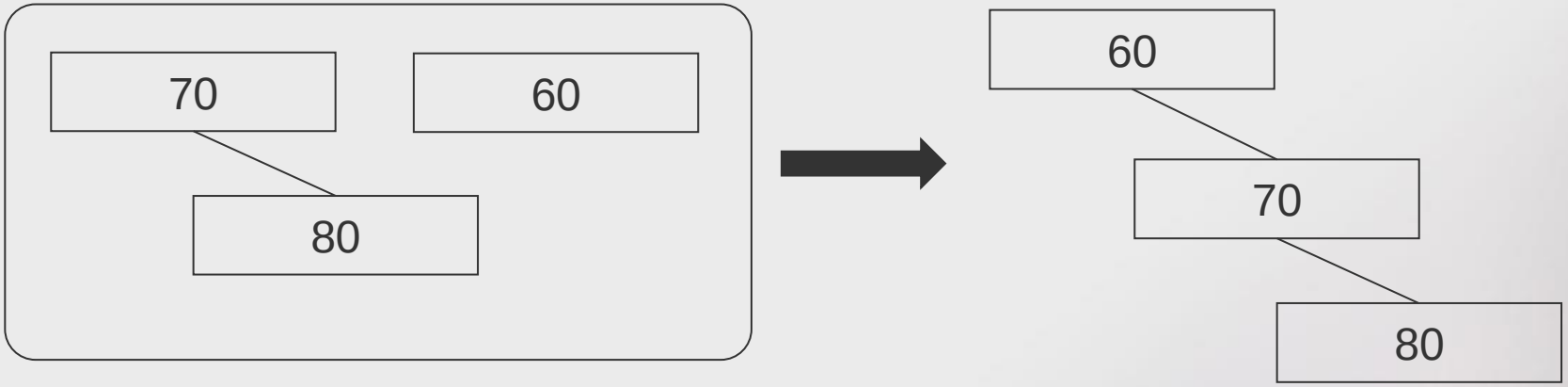
- Inserção de 80





Skew Heap

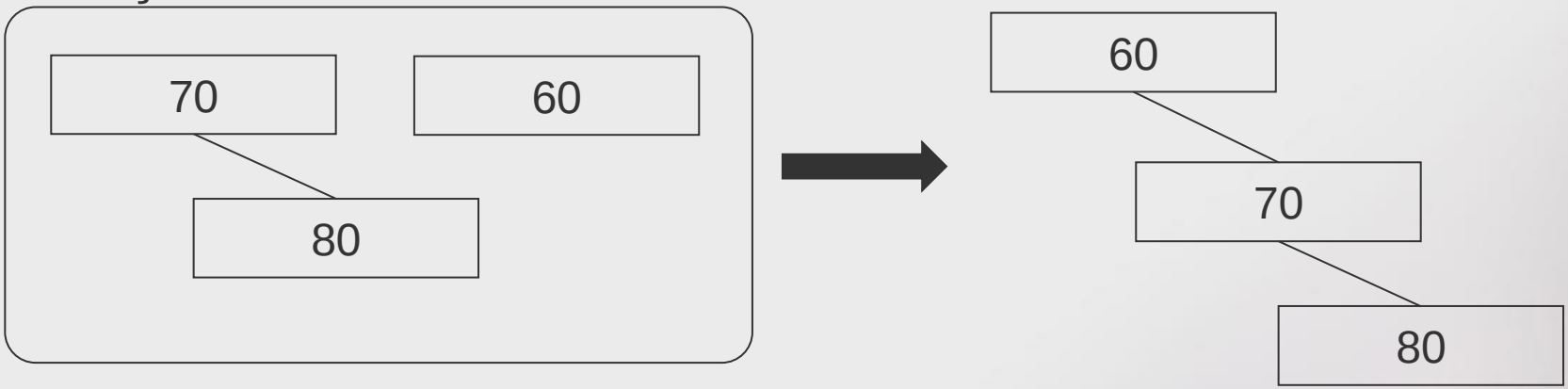
- Vamos ver o que acontece quando inserimos os valores (70, 80, 60) numa àrvore vazia, utilizando a operação de junção “simplista”:
- Inserção de 60



Skew Heap

- Vamos ver o que acontece quando inserimos os valores (70, 80, 60) numa àrvore vazia, utilizando a operação de junção “simplista”:

- Inserção de 60



Dado que o lado esquerdo dos nodos não é alterado, nenhum nodo da *heap* resultante terá descendentes esquerdos. Ou seja, esta será uma lista linear.



Skew Heap

- **Solução?**
- Uma modificação simples ao processo anteriormente descrito assegura que tanto o ramo direito como o ramo esquerdo de cada nodo podem ser afectados pela operação de junção.



Skew Heap

- Em vez de:
- Se *r1≤r2*, a árvore com raiz em *r1* após junção da árvore *r2* com o descendente direito de *r1*.
- Passamos para:
- Se r1≤r2, a árvore com raiz em r1, após junção da árvore r2 com o descendente direito de r1 **seguida de troca dos descendentes direito e esquerdo.**

Ou seja, em cada etapa da junção, trocamos a ordem dos descendentes direitos e esquerdos.

Esta operação de junção é aquilo que define a utilização de uma SKEW Heap



Skew Heap

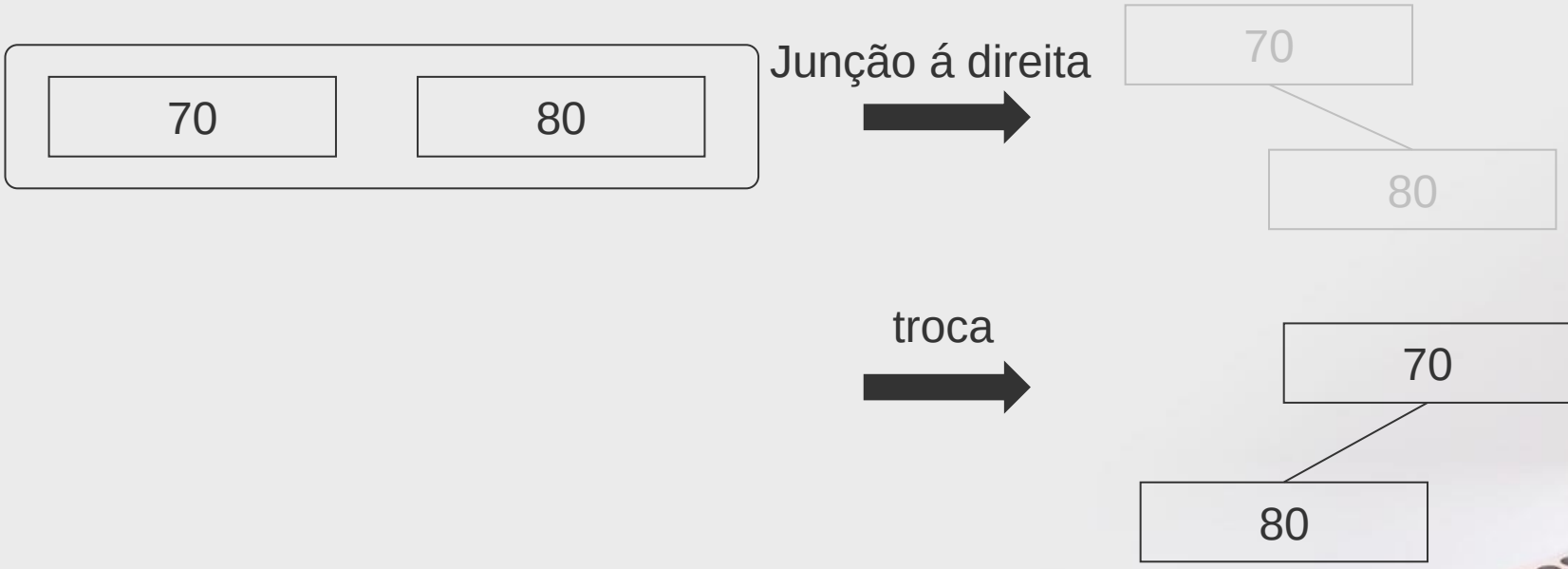
- Vamos ver o que acontece quando inserimos os valores (70, 80, 90) numa àrvore vazia, utilizando a operação de junção de uma *skew heap*:
- Inserção de 70





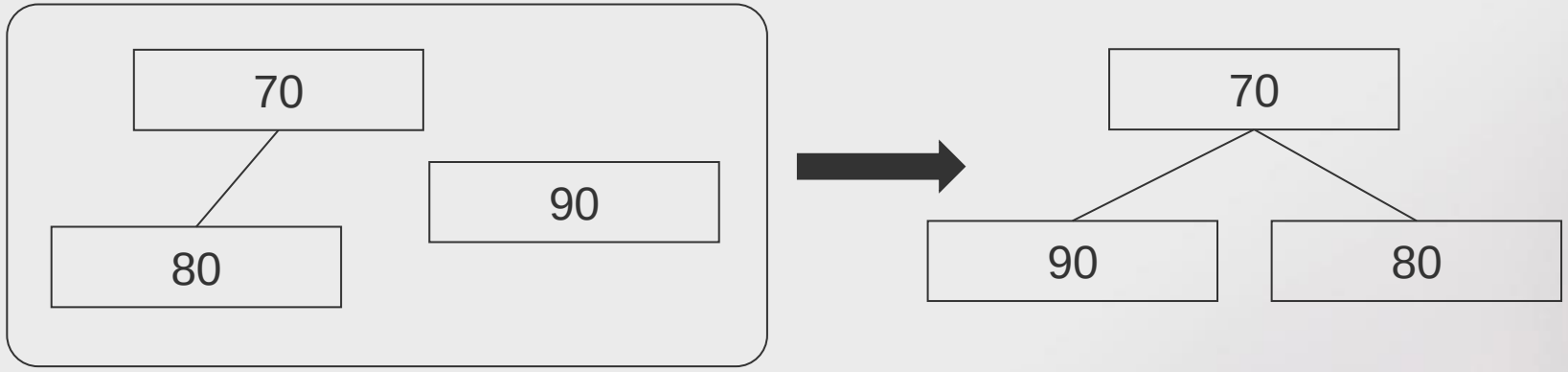
Skew Heap

- Vamos ver o que acontece quando inserimos os valores (70, 80, 90) numa àrvore vazia, utilizando a operação de junção de skew heap:
- Inserção de 80



Skew Heap

- Vamos ver o que acontece quando inserimos os valores (70, 80, 90) numa àrvore vazia, utilizando a operação de junção:
- Inserção de 90



Não se evita completamente a possibilidade de serem criadas árvores degeneradas.
Mas o comportamento resultante é amortizado pelo grande número de operações eficientes que precedem a criação dessa árvore.



Skew Heap - Desempenho

- O custo amortizado para as operações de inserção, remoção do mínimo e junção é:

4 log N

no pior dos casos.



Skew Heap

- A implementação recursiva da junção numa Skew Heap é desaconselhada
  - Apesar da complexidade amortizada ser logaritmica, é linear no pior caso.
  - Isto pode provocar esgotamento da capacidade de recursão nessa circunstância (Stack Overflow).
- É possível criar uma implementação iterativa. Alternativamente, podemos usar outra abordagem…



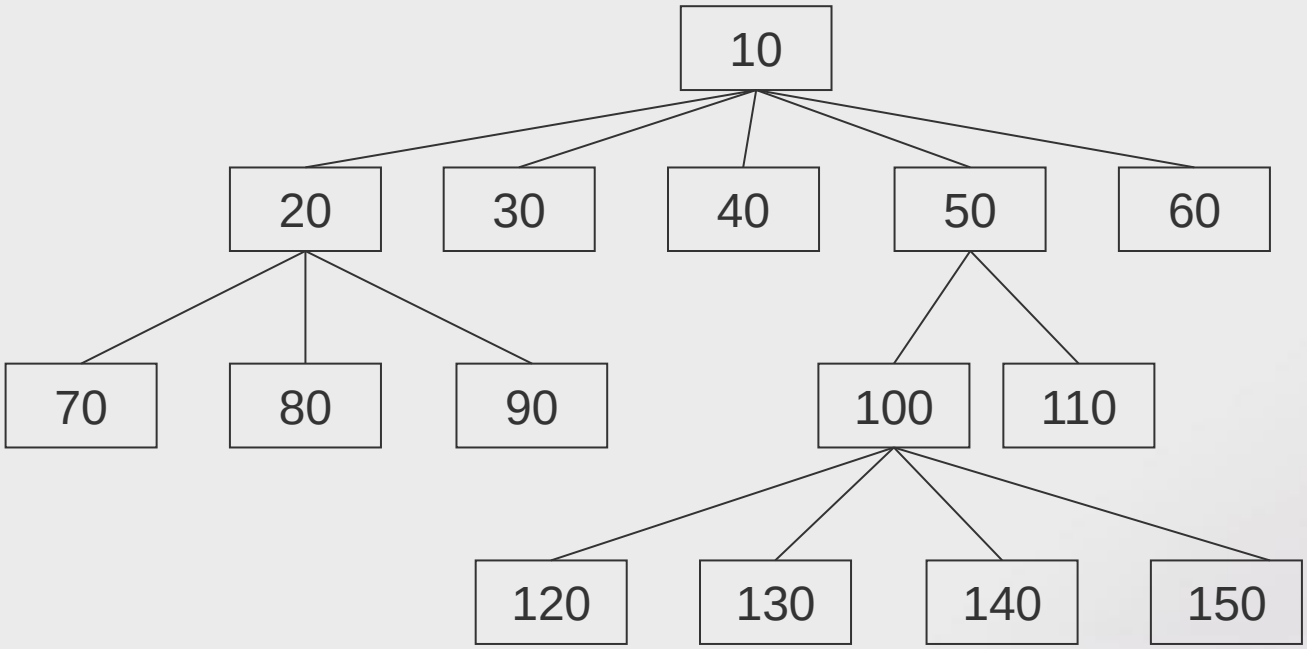
Heap Emparelhada

- Heap M-ária, sem restrições de equilibrio.
- Todas as operações são suportadas em tempo constante, no pior caso ☺, excepto a remoção ☹.
- A remoção demora tempo linear no pior caso, mas sequências de operações têm desempenho logaritmico amortizado.



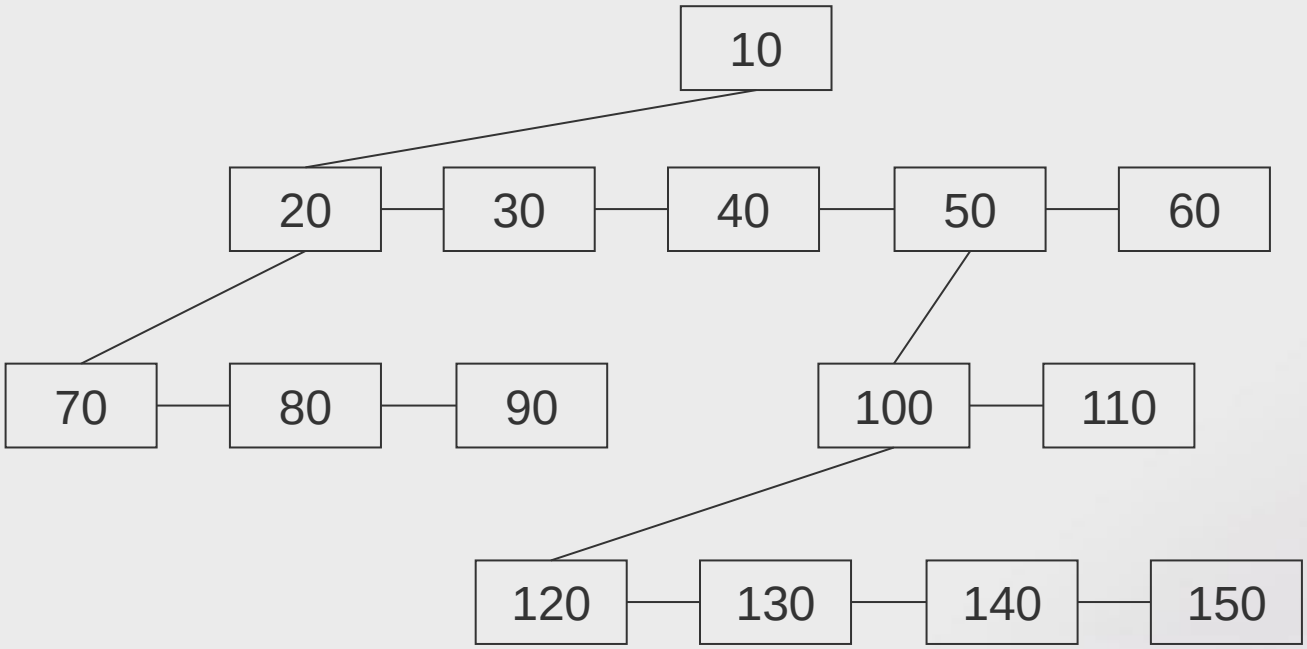
Heap Emparelhada

- Representação conceptual



Heap Emparelhada

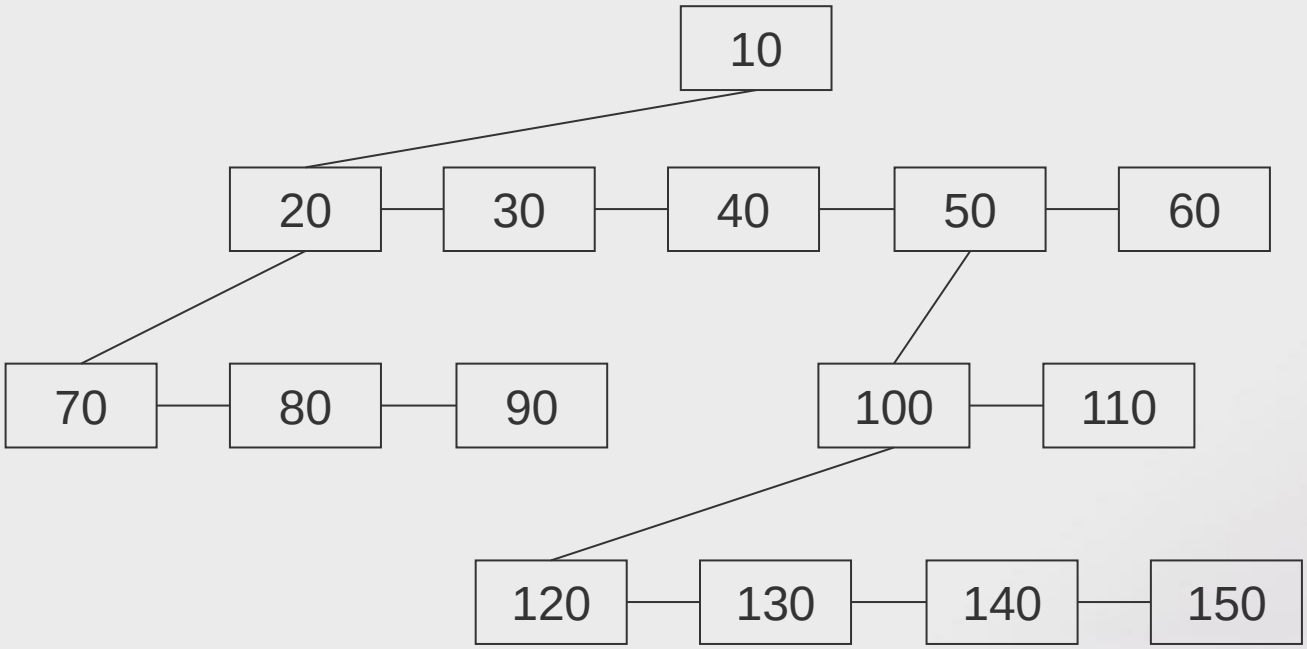
- Implementação com árvore binária
  (com links bidireccionais)



Lado direito: nodo irmão
Lado esquerdo: primeiro descendente

Heap Emparelhada

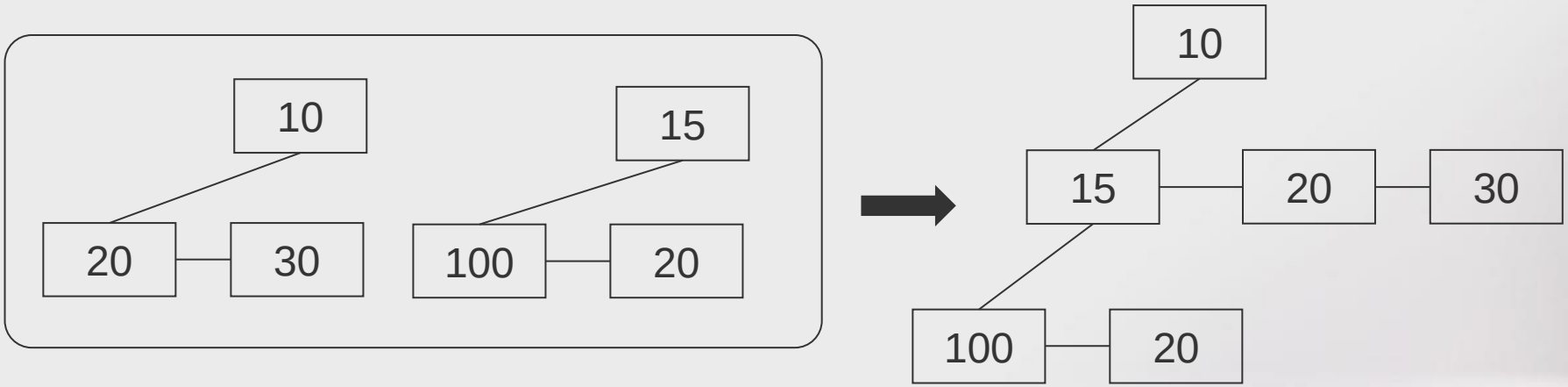
- Implementação com árvore binária
  (com links bidireccionais)



Lado direito: nodo irmão
Lado esquerdo: primeiro descendente

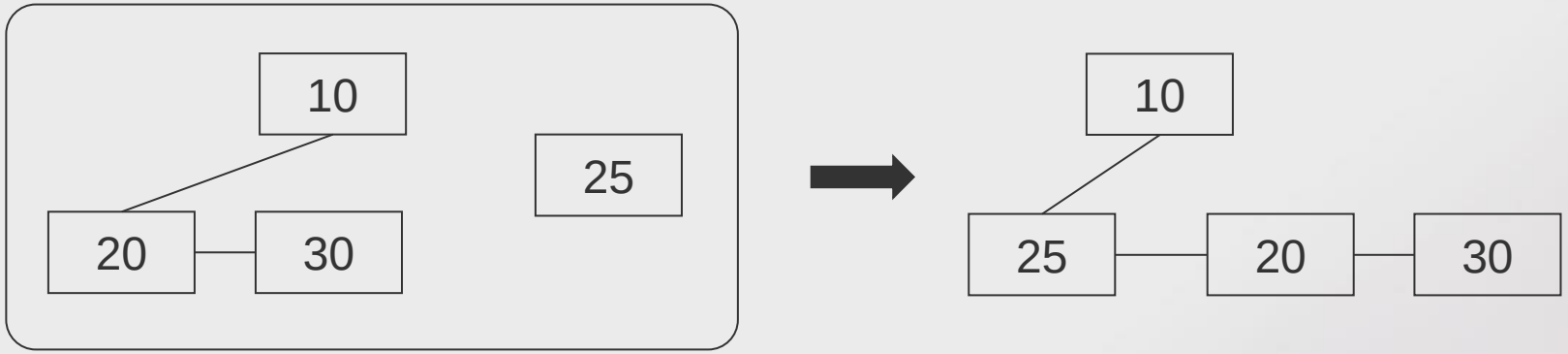
Heap Emparelhada

- Operações:
  - **Junção**: A *heap* com maior raiz passa a ser o primeiro descendente da outra raiz.



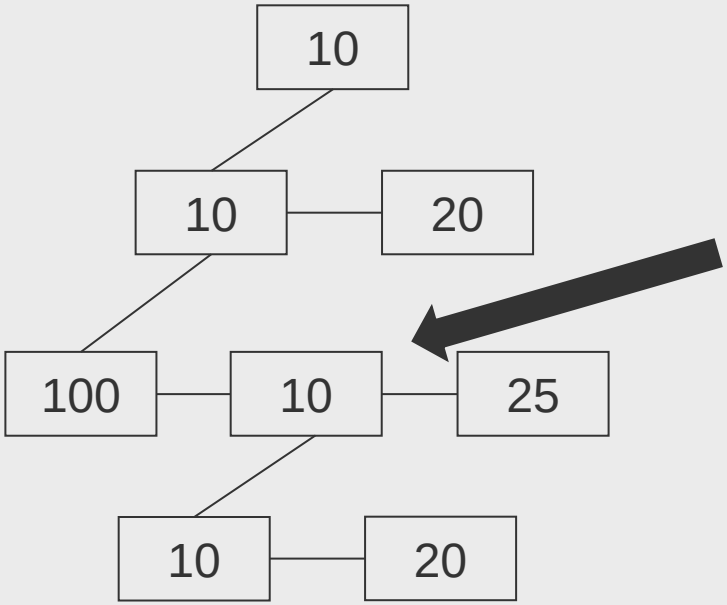
Heap Emparelhada

- Operações:
  - **Inserção**: A inserção é simplesmente a junção com o novo nodo.



Heap Emparelhada

- Operações:
  - **Decrementar Chave**:

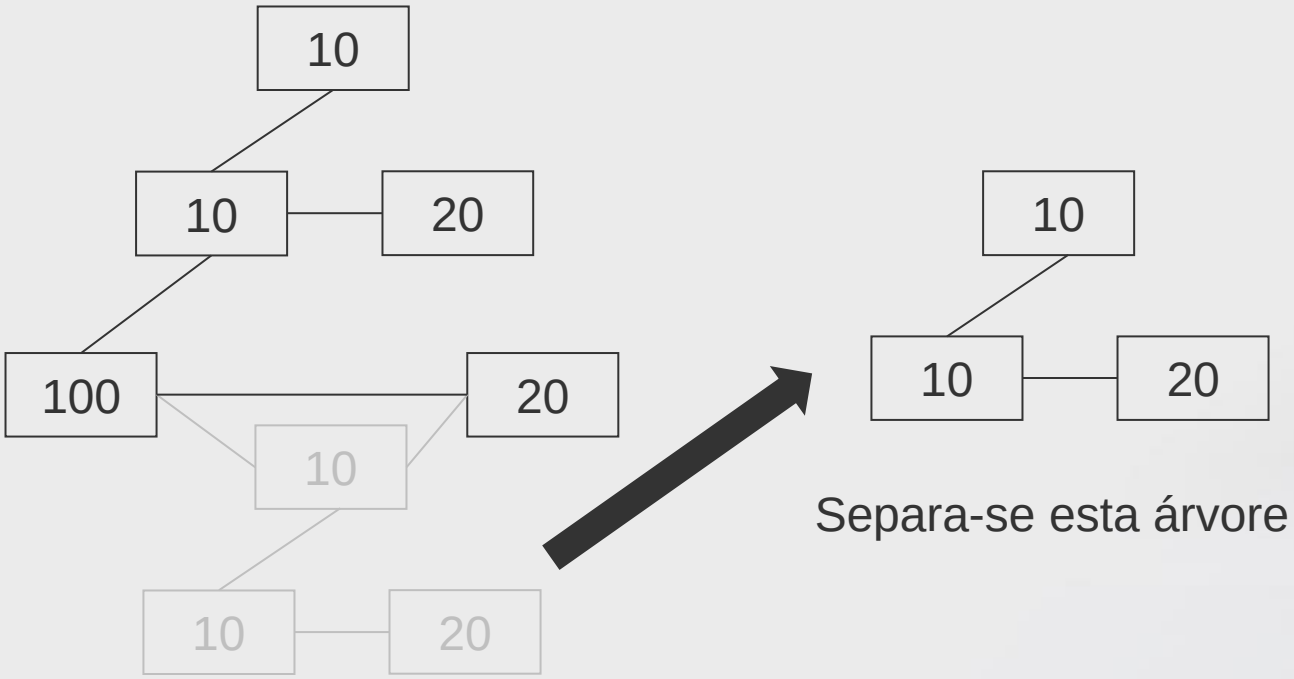


Para decrementar esta chave…



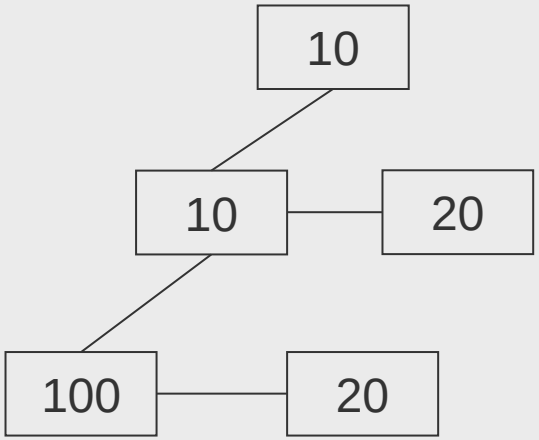
Heap Emparelhada

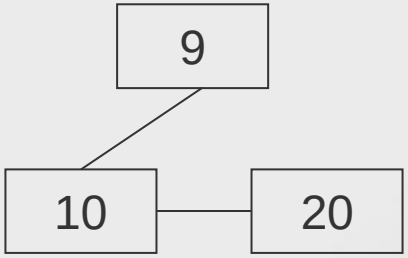
- Operações:
  - **Decrementar Chave**:



Heap Emparelhada

- Operações:
  - **Decrementar Chave**:



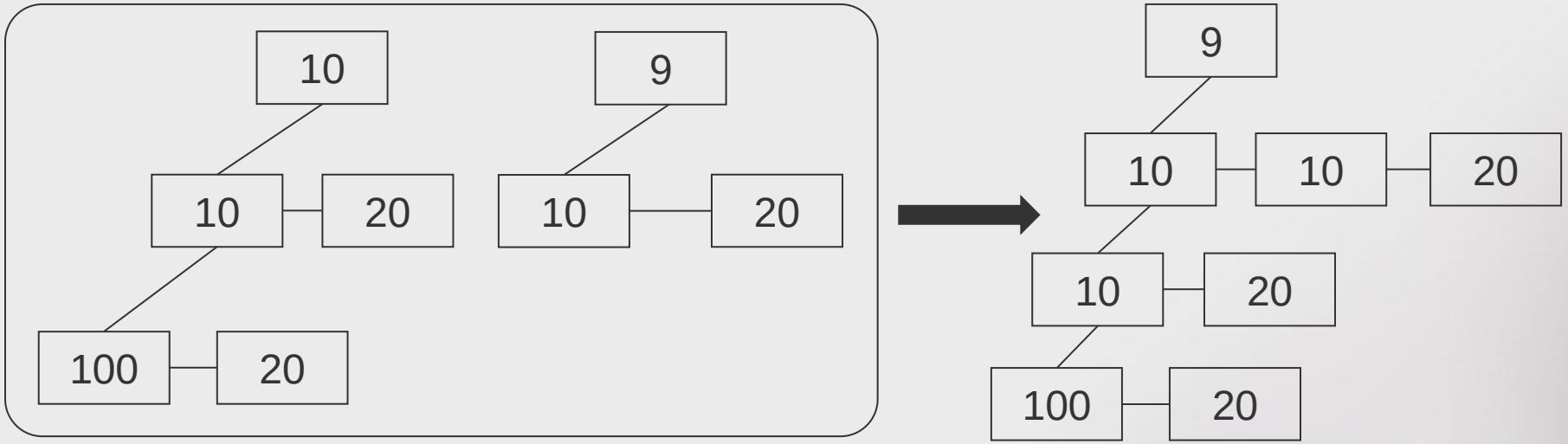


… decrementa-se a chave



Heap Emparelhada

- Operações:
  - **Decrementar Chave**:

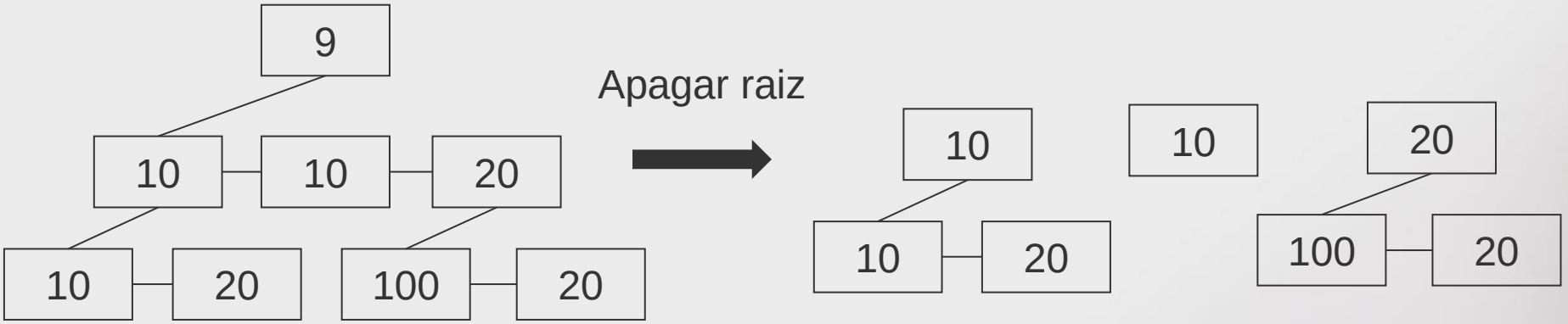


… e juntam-se as árvores resultantes



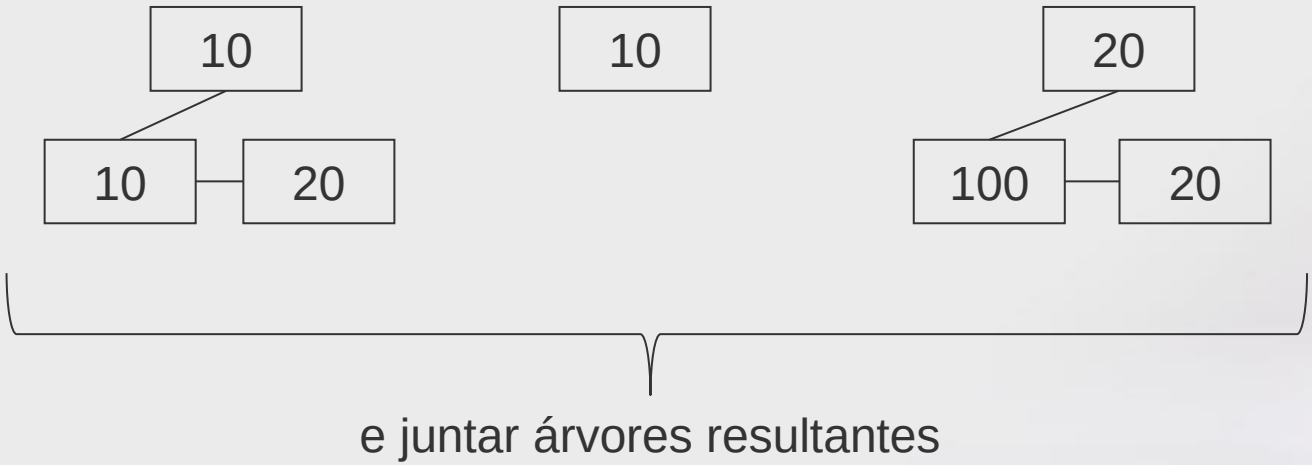
Heap Emparelhada

- Operações:
  - **Remover mínimo**:



Heap Emparelhada

- Operações:
  - **Remover mínimo**:



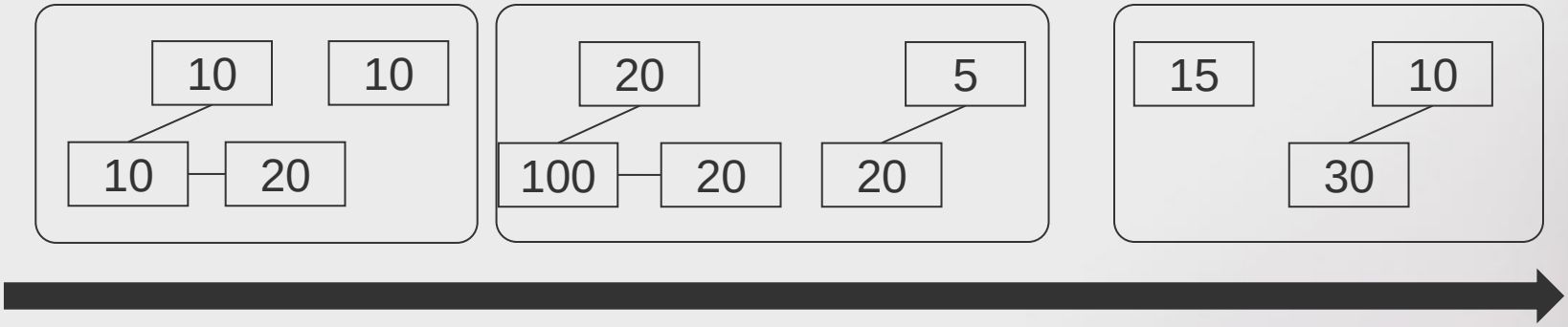
Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Se a sequência de inserção for 1,2,…,N, então todos os nodos depois do primeiro são descendentes directos da raiz.
  - Sendo assim, a remoção demora O(N) no pior caso ☹.
  - *A ordem de junção das subárvores é escolhida de forma a minimizar a reocorrência destas situações.*



Heap Emparelhada

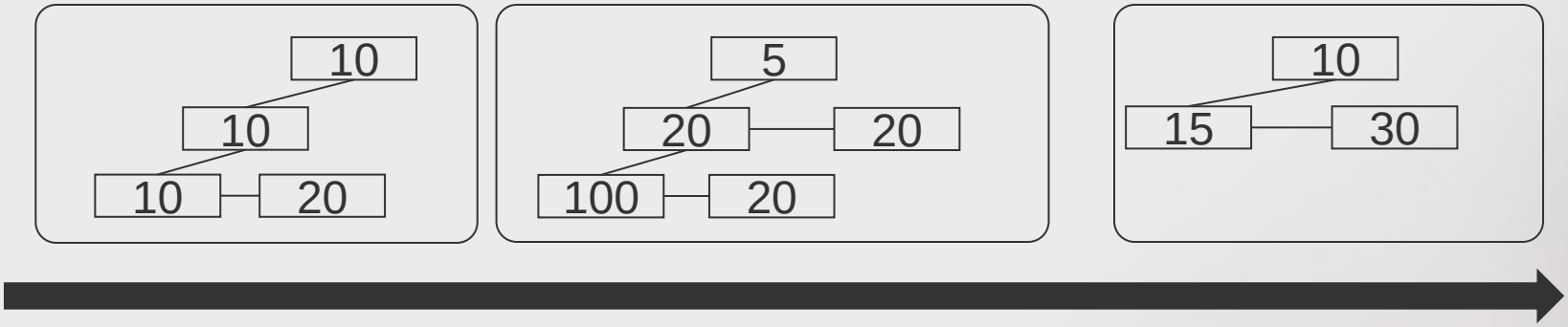
- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar <u>pares</u> da esquerda para a direita*





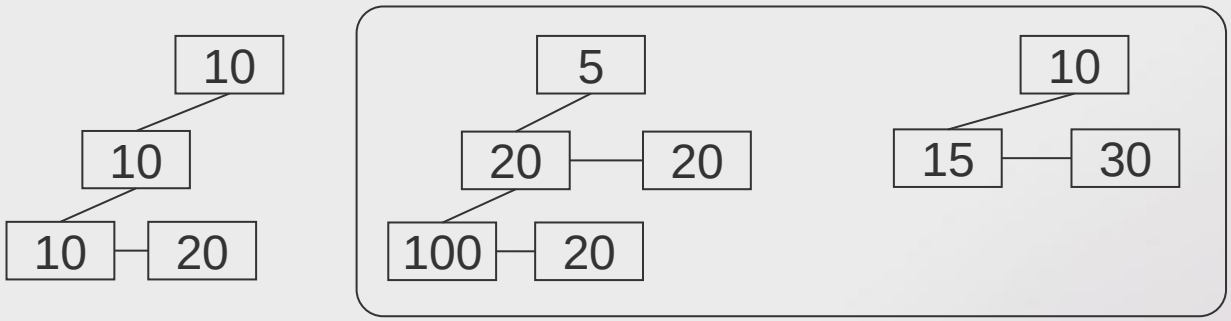
Heap Emparelhada

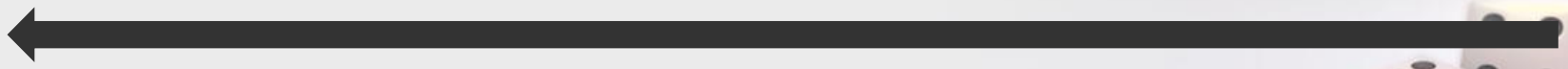
- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar pares da esquerda para a direita*



Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar sequencialmente da direita para esquerda*

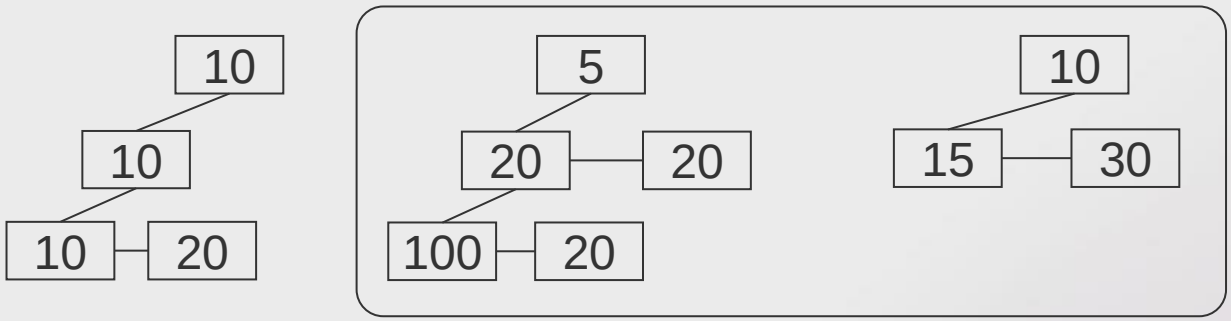


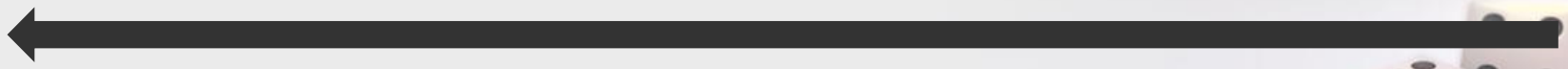




Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar sequencialmente da direita para esquerda*

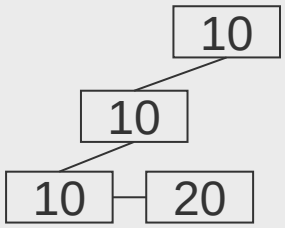


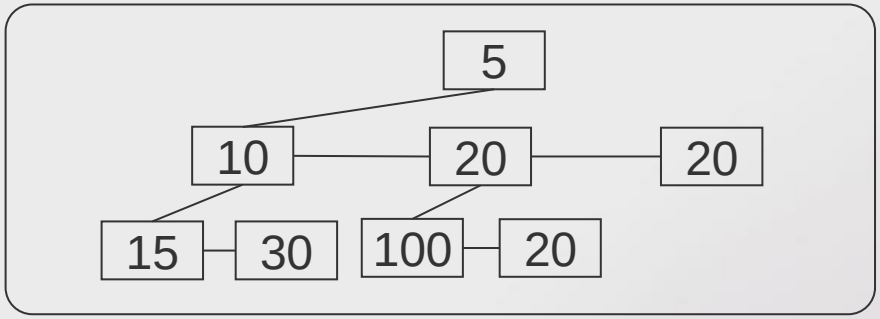


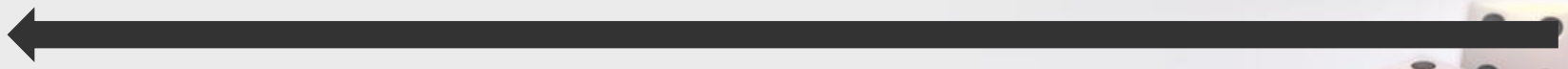


Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar sequencialmente da direita para esquerda*



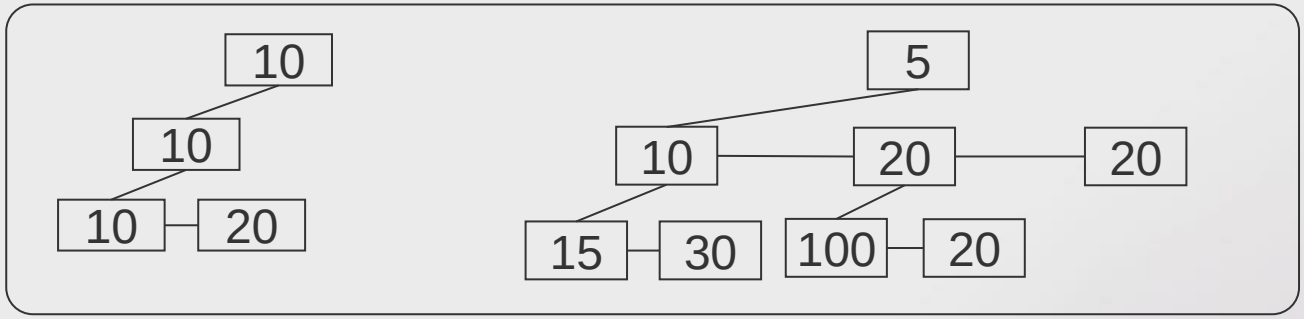


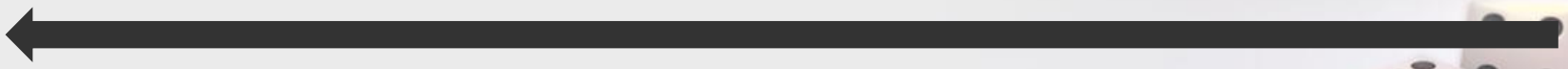




Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar sequencialmente da direita para esquerda*

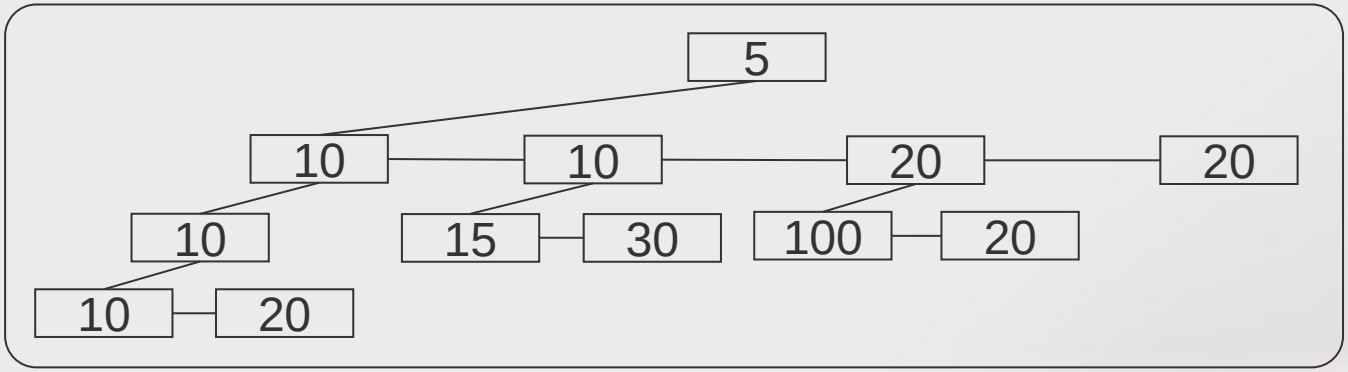


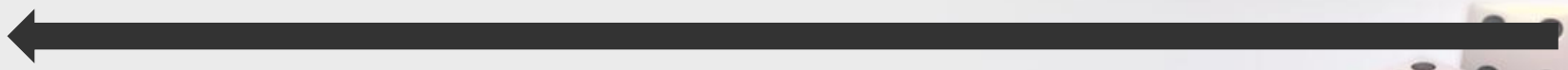




Heap Emparelhada

- Operações:
  - **Remover mínimo**:
  - Junção em dois passos:
    - *Juntar sequencialmente da direita para esquerda*







Heap Emparelhada

- A junção em dois passos assegura comportamento logarítmico amortizado.
  - *O número de descendentes da raiz é controlado.*

